Size-constrained agglomerative clustering

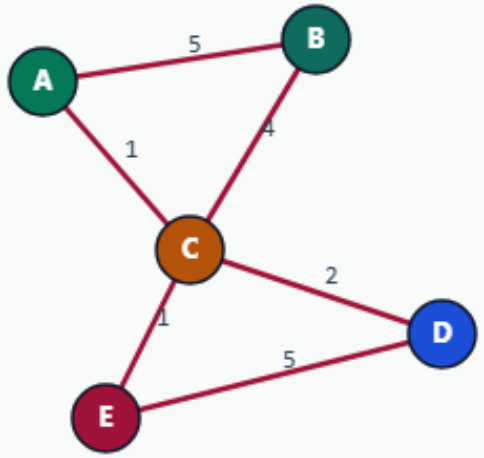
Size caps change which merges are legal

Capacity blocks illegal merges

SIZE-CONSTRAINED AGGLOMERATIVE

Capacity blocks illegal merges

Step 1 / 5 · capacity-rule · module01-05-size-constrained-agglomerative



Explanation

Same greedy heaviest-edge merge, but refuse any pair whose sizes sum above capacity. With capacity=2 on unit nodes, size-3 clusters are illegal.

- Target K=2 with capacity=2
- Legal merges: size ≤ 2
- Unconstrained would absorb C into AB

```
capacity=2
K=2
start: five singletons
```

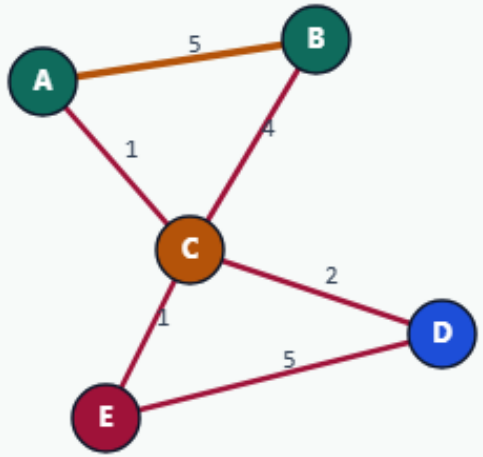
Capacity blocks illegal merges

Merge A–B (still legal)

SIZE-CONSTRAINED AGGLOMERATIVE

Merge A–B (still legal)

Step 2 / 5 · merge-ab · module01-05-size-constrained-agglomerative



Explanation

Heaviest edge A–B at weight 5 merges first. Combined size is 2 — exactly at capacity. C cannot join next.

- First merge identical to unconstrained
- AB size = 2
- Next heaviest legal pair is D–E

```
merge: A–B @ 5
clusters: AB | C | D | E
```

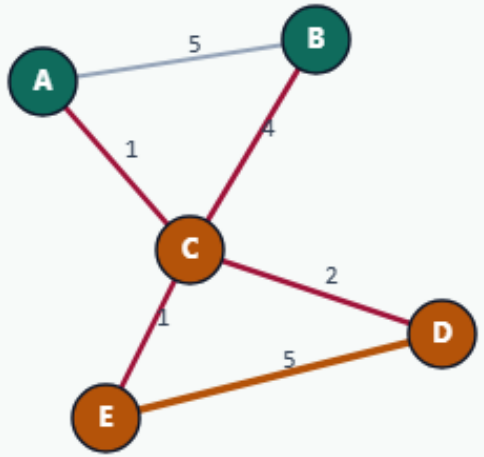
Merge A–B (still legal)

Merge D–E; C stays alone

SIZE-CONSTRAINED AGGLOMERATIVE

Merge D–E; C stays alone

Step 3 / 5 · merge-de · module01-05-size-constrained-agglomerative



Explanation

D–E merges at weight 5. Absorbing C into AB would create size 3 and is rejected, so we stop with three clusters even though target K=2.

- Capacity stalls further merges
- Stuck at K=3 > target
- C is an isolated bridge node

```
merge: D-E @ 5
parts: AB|C|DE
cutsize: 8
```

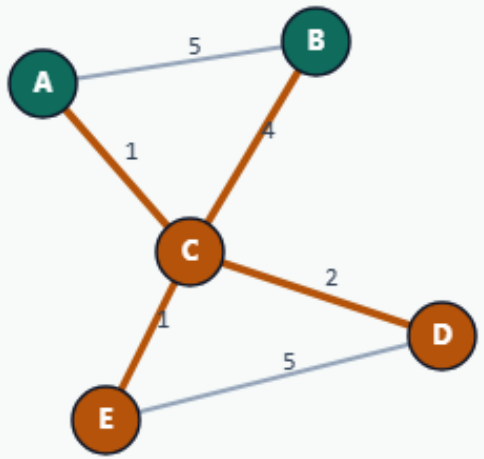
Merge D–E; C stays alone

Result: cutsize 8

SIZE-CONSTRAINED AGGLOMERATIVE

Result: cutsize 8

Step 4 / 5 · cut-8 · module01-05-size-constrained-agglomerative



Explanation

Cut edges include A–C, B–C, C–D, C–E. Capacity cost is +5 versus the unconstrained cut of 3.

- Golden: AB|C|DE, cut=8
- Trade balance for worse cut
- Relax capacity to 3 to recover ABC|DE

```
cutsize: 8
vs unconstrained: 3
```

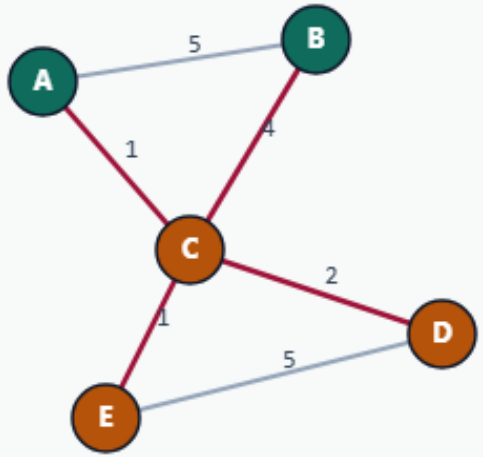
Result: cutsize 8

Why capacity matters in EDA

SIZE-CONSTRAINED AGGLOMERATIVE

Why capacity matters in EDA

Step 5 / 5 · takeaway · module01-05-size-constrained-agglomerative



Explanation

Floorplanning and clustering often cap cluster area. Teaching point: constraints change the greedy path — you may never reach the unconstrained communities.

- Capacity is a hard filter on merges
- May miss global cut optima
- Refinement / multilevel can help later

```
Starter golden: cut=8 with cap=2
```

Why capacity matters in EDA

Browser lab track

- In the browser lab
- Clear the challenges for blocked size-three clusters and the AB|C|DE parts

Implement track

- Run greedy merge with capacity two and confirm cutsize eight with parts AB|C|DE
- Then lift capacity to three and recover the unconstrained ABC|DE cutsize three

Implement track — try these

```
export PYTHONPATH=../common

python ../common/solvers.py examples/tiny_graph.json --k 2 --capacity
2
```

Pitfalls to watch

- Forgetting to check capacity before contracting is the classic bug
- Also watch stale affinities after a blocked merge candidate is skipped

Your turn

- Match the capacity-two golden